

Synthetic Model Generation from Metamodel using Conditional Tabular Generative Adversarial Network

El Abbassia Deba
dept. Computer Science
University Oran 1
Oran, Algeria
abbassia.deba@univ-oran1.dz

Abdelouadoud Sadeuk Ben Abbas
dept. Computer Science
University Oran 1
Oran, Algeria
wadoudsadek1@gmail.com

Karima Berramla
dept. Computer Science
University Oran 1
Oran, Algeria
berramla.karima@gmail.com

Abstract—Building a dataset of models that are metamodel instances for a Model-Driven Engineering (MDE) process using Artificial Intelligence (AI) techniques is now gaining traction. These datasets are rare. For this reason, the availability of such datasets is a challenge in the context of MDE and AI. The central idea of the paper is to use deep learning techniques to build instances of a metamodel that constitute a dataset. Thus, our contribution focuses on creating synthetic instances of a metamodel using a Conditional Tabular Generative Adversarial Network (CTGAN).

Index Terms—Model-Driven Engineering, Model, Metamodel, Domain dictionary, Synthetic data, Generative AI, CTGAN model

I. INTRODUCTION

Artificial intelligence (AI) is evolving rapidly, thanks to advances in hardware and the advent of the big data era. Today, we are witnessing the impact of AI on the software development process. In fact, to date, many research studies predict the direct and long-term involvement of AI in the various phases of the software development lifecycle, from specification, analysis, and design to development, testing and deployment, maintenance, and so on.

Model Driven Engineering (MDE), as a field of software engineering, has not escaped the influence of artificial intelligence in general and machine learning in particular. In fact, a number of research studies have focused on how to simplify the writing of model transformations, which is a key element of MDE. An example of such an approach is Model Transformation By Example (MTBE) [1] [2], which requires developers to provide source models, their corresponding target models, and the mappings between them using a mapping language. The MTBE approach relies on the generation of partial mapping relationships that form the basis of the model transformation. Although MTBE frees the user from having to learn a specific transformation language, the user still has to learn a correspondence language and manually construct/complete the mappings of the generated transformations.

In the context of model transformations, several works have focused on the use of AI techniques to generate model transformations. We cite the use of genetic algorithms [3] and machine learning [4], where the authors propose to adopt a machine learning-based solution to transform an input model into a corresponding output model automatically and without

the need to write transformation-specific code.

Artificial intelligence techniques in general, and those of machine learning in particular, are generally based on the use of massive amounts of data. Indeed, machine learning algorithms, and in particular its supervised learning techniques, enable machines to learn the relationships (mappings) between existing examples in an input-output dataset and then to predict output data based on entirely new input data.

But in the MDE field, unlike image processing or language translation, for example, it is precisely the availability of input data (models) that poses a problem.

To provide a solution to this problem, we propose an approach based on the automatic generation of metamodel-compliant models. These generated models, which are only instances of their metamodel, can constitute a dataset of significant size for use in a machine learning process and also for testing transformation approaches such as in [5]. Our contribution focuses on the automatic generation of a model dataset, based on the integration of metamodel specification in the context of model-driven engineering and the use of CTGAN, a generative model algorithm, to compute a synthetic dataset that can be used to build valid metamodel instances.

The rest of this paper is structured as follows: Contexte, Motivation, and Problem statement are presented in section I. Section II is devoted to present the background of the study. Detailed description and evaluation of the proposed approach are defined in section III and IV. In section V we present related work and finally, section VI concludes this paper.

II. PRELIMINARIES

The proposed approach is based on the integration of two domains: model-driven engineering and generative machine learning. In the following, we introduce these two fields.

A. Model-Driven Engineering

Model-driven engineering (MDE) is an approach to software engineering in which the model is considered the central concept. Its main goal is to generate all or part of an application's code from models. Up to now, this approach is an active research topic that makes intensive use of metamodels and model transformations.

Generally, the practice of metamodeling is the specification

Models are used to model a system and to develop, maintain, and evolve the software by performing model transformations. In a broad sense, the MDE paradigm proposes to unify all aspects of the life cycle process by using the notions of system, model, metamodel, and metametamodel [6].

Another trend is that the MDE paradigm is expected to extend beyond domain-specific languages (DSLs) and code generation, aiming for a comprehensive integration of engineering and architectural capabilities. This could provide a more principled and transparent approach to systems engineering [8].

Generally key concepts of generative machine learning include:

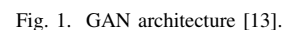
- **Latent Space:** A lower-dimensional representation of data in which similar items are close together.
- **Sampling:** The process of drawing new examples from the learned distribution.
- **Conditional Generation:** The generation of samples based on specific input conditions or attributes.
- **Likelihood Estimation:** Measuring how well the model represents the true data distribution.

Another technique is unsupervised machine learning models

Conditional Tabular GAN (CTGAN) is a specific type of GAN designed to generate synthetic tabular data that preserves the statistical properties of the original dataset [12]. It addresses some of the challenges of applying standard GANs to tabular data, which are:

- The key innovations in CTGAN include:

- Key differences in CTGAN’s architecture and learning process make it more capable of generating high-quality synthetic tabular data than traditional GAN models. These changes enable CTGAN to capture the complex statistical properties and data types found in real-world structured datasets.



The summary of the followed approach is described in the figure 2.

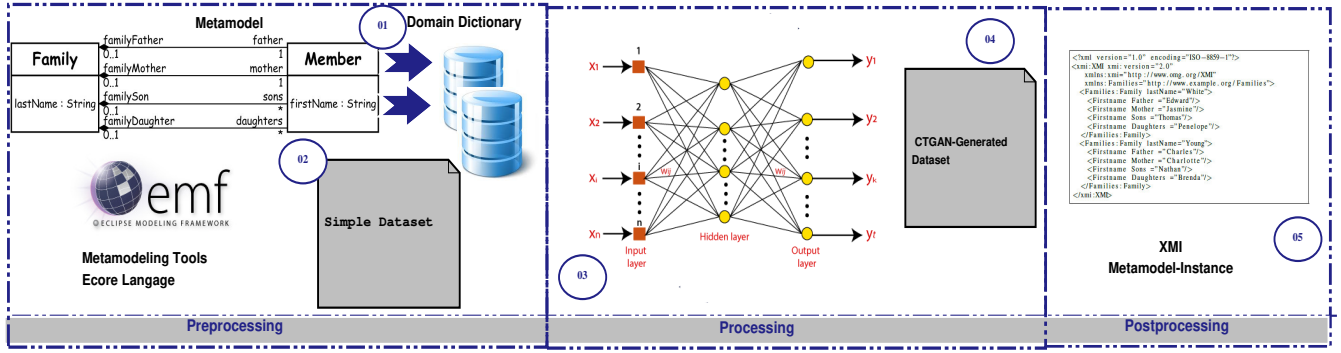


Fig. 2. Model Generation process.

As an illustrative example, we use a metamodel borrowed from [14]. The chosen metamodel comes from a use case of a model-to-model transformation (see Figure 3). It defines a collection of families. Each family has a name and consists of members: a father, a mother, several sons and several daughters. Each family member has a first name.

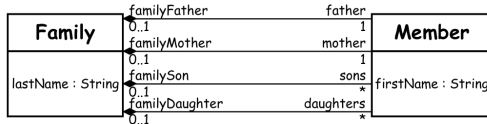


Fig. 3. Families metamodel [14].

Generally, the approach consists of three steps: a preprocessing step, a processing step, and a postprocessing step.

A. Preprocessing step: Combining a metamodel and domain dictionary to prepare the table header of CTGAN

The choice between using a metamodel or a domain dictionary often depends on the specific use case, with metamodels being more common in model-driven engineering approaches and domain dictionaries being more suitable for data generation and validation tasks.

Both metamodels and domain dictionaries are used to define and structure information within a given domain. They provide a framework for understanding and organizing data, which is crucial for generating synthetic datasets.

Let M be a set of models, and L be a formal language used to express these models. A metamodel MM can be formally defined as: $MM = (C, R, A, Const)$ Where:

- C is a set of concepts (or metaclasses) that define the types of elements in the models.
- R is a set of relationships between concepts.
- A is a set of attributes that describe properties of the concepts.
- $Const$ is a set of constraints that define rules and restrictions on the models.

As a metamodel is used and refined over time, it can become more comprehensive and serve to obtain a dictionary for the

concepts and relationships within a specific domain [15]. Table I is an excerpt of the Ecore presentation of the metamodel example (Fig. 3) that is one of concrete syntaxes to serialize metamodels [16].

TABLE I
EXCERPT OF ECORE FAMILIES METAMODEL.

```
<xmi:XMI xmi:version="2.0" xmlns:xmi="http://www.omg.org/XMI"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:ecore="http://www.eclipse.org/emf/2002/Ecore">
  <ecore:EPackage name="Families">
    <eclassifiers xsi:type="ecore:EClass" name="Family">
      <estructuralfeatures xsi:type="ecore:EAttribute"
name="lastName" ordered="false"
unique="false" lowerBound="1"
eType="/1/String"/>
      <estructuralfeatures xsi:type="ecore:EReference"
name="father" ordered="false"
lowerBound="1" eType="/0/Member"
containment="true"
eOpposite="/0/Member/familyFather"/>
      ...
    </eclassifiers>
  </ecore:EPackage>
  <ecore:EPackage name="PrimitiveTypes">
    <eclassifiers xsi:type="ecore:EDatatype" name="String"/>
  </ecore:EPackage>
</xmi:XMI>
```

A domain dictionary focuses more on the terminology and definitions within a domain. It provides a detailed list of terms and their meanings but does not necessarily define the relationships or rules for how these terms interact.

A domain dictionary is a formal representation of domain-specific knowledge, concepts, and rules that can be used to guide data generation, validation, or analysis processes. Let's define it more formally:

A domain dictionary D is a tuple (C, A, R, F) where:

- 1) C is a set of concepts or entities in the domain.
- 2) A is a set of attributes associated with those concepts.
- 3) R is a set of relationships between concepts.
- 4) F is a set of functions or rules that govern the domain.

This formal definition provides a structured way to represent the domain knowledge. It can be used to guide the data

generation process in CTGAN or in other synthetic data generation methods and to validate the generated data to ensure it conforms to domain rules.

Due to limited space, we explain the idea of mapping a metamodel to a domain dictionary. The purpose of this mapping is to prepare the head table of the CTGAN model.

Here's a more detailed mapping using the provided Ecore metamodel (see Table II):

TABLE II
MAPPING FAMILIES METAMODEL TO DOMAIN DICTIONARY.

Ecore Metamodel	Domain Dictionary
EClass: Family - EAttribute: lastName - EReference: father - EReference: mother - EReference: sons - EReference: daughters	C: Family concept A: (family_last_name, string, not null) R: (Family, Member, one-to-one, "has father") R: (Family, Member, one-to-one, "has mother") R: (Family, Member, one-to-many, "has sons") R: (Family, Member, one-to-many, "has daughters")
EClass: Member - EAttribute: firstName - EReference: familyFather - EReference: familyMother - EReference: familySon - EReference: familyDaughter	C: Member concept A: (member_first_name, string, not null) R: (Member, Family, many-to-one, "is father of") R: (Member, Family, many-to-one, "is mother of") R: (Member, Family, many-to-one, "is son of") R: (Member, Family, many-to-one, "is daughter of")
EDataType: String	(implicitly defined in attribute types)

The combination of a metamodel and a domain dictionary provides a solid foundation for the preparation of a table header for CTGAN model. This approach uses the structural information of the metamodel and enriches it with the detailed knowledge of the domain dictionary.

The table header structure for the CTGAN model is derived from a metamodel and a domain dictionary. The first column of Table III shows a view of the table header structure derived for the CTGAN model associated with the example in Fig. 3. The CTGAN table consists of a set of columns. The specification for each column consists of its name, its type, and a set of possible values according to the associated data type. These values are taken from the domain dictionary. In general, depending on the associated type, the data in each column is subject to constraints.

TABLE III
EXCERPT OF A DOMAIN DICTIONARY

Key	Type	Value
Family_lastname	str	march, sailor, Anderson, Brown, Clark, Davis, Evans, Garcia, Harris, Johnson, King, Lewis, Martinez, Nelson, Parker, Robinson, Smith, Taylor, Walker, White, Young, Wright
Member_FirstnameFather	str	Jim, Peter, Kevin, James, Isaac, Henry, George, Frank, Edward, David, Charles, Benjamin
Member_FirstnameMother	str	Cindy, Jakie, Katherine, Jasmine, Isabelle, Hannah, Grace, Fiona, Emily, Diana, Charlotte, Bianca
Member_FirstnameDaughters	str	Brenda, Kelly, Abigail, Victoria, Sophia, Rebecca, Quinn, Penelope, Olivia, Natalie, Maria, Leah
Member_FirstnameSons	str	David, Dylan, Brandon, Alexander, Liam, Michael, Nathan, Oliver, Patrick, Quentin, Robert, Samuel, Thomas

B. Processing Step: Using CTGAN for generating tabular synthetic Data

The processing stage consists of executing the CTGAN model and providing it with the structure of the table header of the dataset to be generated, as well as a data sample. As we mentioned earlier, the table header is derived from the metamodel. CTGAN can then generate synthetic data that closely mimics the structure and characteristics of the valid data sample. This approach ensures that the generated data retains its relevance and usefulness for further analysis. The table IV shows an excerpt of the dataset generated by CTGAN from the metamodel of the example used.

TABLE IV
EXCERPT OF CTGAN RESULT FOR FAMILIES METAMODEL.

Family last-name	Member First-name Daughters	Member First-name Father	Member First-name Mother	Member First-name Sons
White	Penelope	Edward	Jasmine	Thomas
Young	Brenda	Charles	Charlotte	Nathan
Sailor	Rebecca	Edward	Fiona	Dylan
Anderson	Abigail	George	Diana	Brandon
Sailor	Victoria	Frank	Isabelle	Quentin
Parker	Penelope	Benjamin	Emily	Nathan
Parker	Natalie	Benjamin	Hannah	Alexander
Taylor	Penelope	Isaac	Isabelle	Brandon
Harris	Maria	Charles	Fiona	Michael
March	Quinn	James	Diana	Quentin

C. Postprocessing Step: Generating Instance Metamodel

Building a metamodel instance from a metamodel-generated dataset is a multi-step process. We propose an algorithm (cf. algorithm 1) that allows to construct an XMI model [17] from the CTGAN results, which is just an instance of the metamodel.

First, the metamodel is parsed to obtain its structure and constraints, and then the dataset is evaluated to determine its contents. An empty XMI instance model is then created using the parsed metamodel, and data points from the dataset are extracted and assigned to the models elements. The extracted data points are then entered into the XMI metamodel instance, which is then evaluated to ensure compatibility with the metamodels structure and restrictions. Finally, the approved XMI metamodel instance is serialized into the XMI format, preparing it for use in a variety of applications.

This process of generating synthetic metamodel instances improves data understanding, validation, analysis, and model-driven development. It can be used in domains such as data science, software engineering, and business intelligence to facilitate data integration, analysis, model-driven development, and validation.

Algorithm 1 Generate XMI Instance Metamodel

Require: Metamodel Specification, Dataset

Ensure: XMI Model

- 1: PARSE METAMODEL(Metamodel Specification) {Parse and load classes, attributes, relationships, and constraints from the metamodel specification.}
- 2: ANALYZE DATASET(Dataset) {Understand the structure and contents of the dataset generated from the metamodel.}
- 3: INITIALIZE XMI MODEL {Create an empty structure for the XMI instance metamodel based on the parsed metamodel.}
- 4: EXTRACT DATA POINTS FROM DATASET {// Identify and extract data points (instances, attributes, relationships) from the dataset that correspond to the metamodel.}
- 5: POPULATE XMI MODEL {// Populate the initialized XMI instance model with data points extracted from the dataset.}
- 6: VALIDATE XMI MODEL {// Validate the populated XMI instance model against the metamodels structure and constraints.}
- 7: SERIALIZE TO XMI FORMAT {// Convert the validated XMI instance model into the XMI format.}
- 8: **return** Serialized XMI Instance Metamodel

As an example of the result of the post-processing step, Table V shows, from the metamodel used (see Figure 3), an example of a generated metamodel instance that is extracted from the dataset computed with CTGAN.

TABLE V
XMI GENERATED INSTANCE METAMODEL.

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<xmi:XMI xmi:version="2.0"
  xmlns:xmi="http://www.omg.org/XMI"
  xmlns:Families="http://www.example.org/Families">
  <Families:Family lastName="White">
    <Firstname Father ="Edward"/>
    <Firstname Mother ="Jasmine"/>
    <Firstname Sons ="Thomas"/>
    <Firstname Daughters ="Penelope"/>
  </Families:Family>
  <Families:Family lastName="Young">
    <Firstname Father ="Charles"/>
    <Firstname Mother ="Charlotte"/>
    <Firstname Sons ="Nathan"/>
    <Firstname Daughters ="Brenda"/>
  </Families:Family>
</xmi:XMI>
```

IV. DISCUSSION

Our approach provides a new idea that automates the modeling steps by using a generative machine learning technique such as CTGAN. Fig. 4 shows the confusion matrix that helps assess model performance by revealing how well predictions match reality. The classification model's performance is:

- **True Negatives (TN):** 6600 instances correctly identified as class 0.
- **False Positives (FP):** 0 instances incorrectly identified as class 1.
- **False Negatives (FN):** 451 instances incorrectly identified as class 0.
- **True Positives (TP):** 49 instances correctly identified as class 1.

This suggests a high number of true negatives and a low number of true positives, indicating potential issues with

sensitivity or class imbalance.

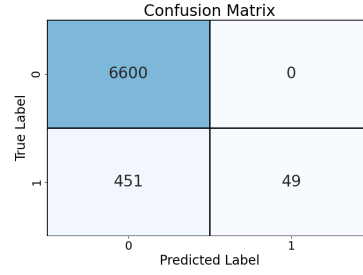


Fig. 4. Confusion Matrix.

Fig. 5 shows the balanced accuracy score of a classification model, which is approximately 0.5. This score indicates a moderate level of accuracy, taking into account class imbalances. Balanced accuracy, the average of sensitivity and specificity, provides a more comprehensive measure in scenarios with unbalanced classes.

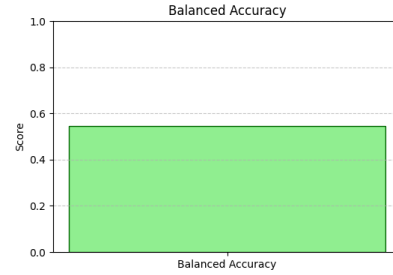


Fig. 5. Balanced Accuracy.

For small datasets, it is clear that automatic generation of metamodel instances can be easily achieved with a simple metamodel containing only concepts and relationships between the defined concepts. In the future, we will consider a metamodel with embedded OCL constraints. The generation process will be improved with other theories and tools.

V. RELATED WORK

Synthetic metamodel instances are crucial for testing, validating, and benchmarking model transformation tools and other model-driven engineering applications. Several works have focused on the problem of automatically generating metamodel instances. We can categorize these works according to the technique or approach used. The first approach is known as constraint-based generation. The authors in [18] present a method for verifying UML/OCL class diagrams using constraint programming, which can be adapted for instance generation. Semerth et al. [19] introduce a technique for evaluating graph constraints over partial models by rewriting constraints, useful for incremental instance generation. Another paper [20] proposes the use of Satisfiability Modulo

Theories (SMT) solvers to generate metamodel instances by exploiting attributed type graphs.

Another graph-based approach treats metamodels as graphs and uses graph transformation rules to generate instances with specific structural properties. As an example, we cite the paper [21], which presents a method for automatically deriving specifications for model-editing operations from metamodels that can be used for instance generation.

Search-based generation is an approach that uses metaheuristic search algorithms (e.g. genetic algorithms) to generate metamodel instances. Shelburg et al. [22] introduce a multi-objective approach to regression testing for model transformations, which can be adapted for instance generation. Fleck et al. [3] explore the integration of search-based optimization techniques for instance generation.

The authors of [23] present techniques for generating realistic test models for model processing tools, combining template-based approaches with other strategies.

A recent trend in the automatic generation of metamodel instances is the integration of AI techniques for more realistic instance generation or a focus on domain-specific instance generation. [24] and [25] are examples.

VI. CONCLUSION

Given that MDE is a model-centered approach, the availability of models that are instances of a metamodel is problematic in the context of model-driven engineering in conjunction with artificial intelligence. This is because software engineering is essentially based on models for the design and more or less automatic generation of applications.

The proposed approach provides a flattened, denormalized representation of the metamodel structure for use with CTGAN. It captures only certain essential relationships and constraints of the Ecore metamodel while adapting it to a format suitable for tabular data generation. To generalize the method, we need to take into account all the types of relationships and OCL constraints that may be present in a complex metamodel.

As perspective work, we propose to consider the verification of complex OCL constraints by using appropriate techniques. We also propose to apply other machine learning methods to perform a comparative study aiming at selecting, from a set of properties, the appropriate method for the modeling step in the MDE context.

REFERENCES

- [1] G. Kappel, P. Langer, W. Retschitzegger, W. Schwinger, and M. Wimmer, "Model transformation by-example: a survey of the first wave," *Conceptual Modelling and Its Theoretical Foundations: Essays Dedicated to Bernhard Thalheim on the Occasion of His 60th Birthday*, pp. 197–215, 2012.
- [2] K. Berramla, E. A. Deba, and D. Benhamamouch, "Model transformation generation a survey of the state-of-the-art," in *2016 International Conference on Information Technology for Organizations Development (IT4OD)*. IEEE, 2016, pp. 1–6.
- [3] M. Fleck, J. Troya, and M. Wimmer, "Marrying search-based optimization and model transformation technology," *Proc. of NasBASE*, pp. 1–16, 2015.
- [4] L. Burgueno, J. Cabot, S. Li, and S. Gérard, "A generic LSTM neural network architecture to infer heterogeneous model transformations," *Software and Systems Modeling*, vol. 21, no. 1, pp. 139–156, 2022.
- [5] K. Berramla, E. A. Deba, A. E. H. Benyamina, R. Touam, Y. Brahimi, and D. Benhamamouch, "Formal concept analysis for specification of model transformations," in *2017 First International Conference on Embedded & Distributed Systems (EDiS)*, 2017, pp. 1–6.
- [6] M. D. Del Fabro, J. Bézin, F. Jouault, E. Breton, and G. Gueltas, "AMW: a generic model weaver," in *1 ere Journées sur l'Ingénierie Dirigée par les Modèles (IDM05)*, 2005, pp. 105–114.
- [7] L. Burgueno, D. Di Ruscio, H. Sahraoui, and M. Wimmer, "The Past, Present, and Future of Automation in Model-Driven Engineering," *arXiv preprint arXiv:2405.18539*, 2024.
- [8] B. Steffen, F. Gossen, S. Naujokat, and T. Margaria, "Language-driven engineering: from general-purpose to purpose-specific languages," *Computing and Software Science: State of the Art and Perspectives*, pp. 311–344, 2019.
- [9] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," *Advances in neural information processing systems*, vol. 27, 2014.
- [10] M. Durgadevi et al., "Generative Adversarial Network (GAN): A general review on different variants of GAN and applications," in *2021 6th International Conference on Communication and Electronics Systems (ICCSES)*. IEEE, 2021, pp. 1–8.
- [11] L. Mescheder, S. Nowozin, and A. Geiger, "Adversarial variational bayes: Unifying variational autoencoders and generative adversarial networks," in *International conference on machine learning*. PMLR, 2017, pp. 2391–2400.
- [12] L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni, "Modeling tabular data using conditional GAN," *Advances in neural information processing systems*, vol. 32, 2019.
- [13] T. team, "Synthetic data," <https://www.turing.com/kb/synthetic-data-generation-techniques>, accessed on 03/03/2024.
- [14] Eclipse/EMF, "ATL Transformations," <https://eclipse.dev/atl/atlTransformations/>, accessed on 02/06/2024.
- [15] P. Atzeni, G. Gianforme, and P. Cappellari, "A Universal Metamodel and Its Dictionary," *T. Large-Scale Data- and Knowledge-Centered Systems*, vol. 1, pp. 38–62, 01 2009.
- [16] Eclipse Foundation, "Eclipse Modeling Framework," <https://eclipse.dev/modeling/emf/>.
- [17] OMG, "XML Metadata Interchange (XMI) Specification," <https://www.omg.org/spec/XMI/2.5.1/PDF>.
- [18] J. Cabot, R. Clarisó, and D. Riera, "On the verification of UML/OCL class diagrams using constraint programming," *Journal of Systems and Software*, vol. 93, pp. 1–23, 2014.
- [19] O. Semeráth and D. Varró, "Graph constraint evaluation over partial models by constraint rewriting," in *Theory and Practice of Model Transformation: 10th International Conference, ICMT 2017, Held as Part of STAF 2017, Marburg, Germany, July 17-18, 2017, Proceedings 10*. Springer, 2017, pp. 138–154.
- [20] H. Wu, R. Monahan, and J. F. Power, "Exploiting attributed type graphs to generate metamodel instances using an SMT solver," in *2013 International Symposium on Theoretical Aspects of Software Engineering*. IEEE, 2013, pp. 175–182.
- [21] T. Kehrer, G. Taentzer, M. Rindt, and U. Kelter, "Automatically deriving the specification of model editing operations from meta-models," in *Theory and Practice of Model Transformations: 9th International Conference, ICMT 2016, Held as Part of STAF 2016, Vienna, Austria, July 4-5, 2016, Proceedings 9*. Springer, 2016, pp. 173–188.
- [22] J. Shelburg, M. Kessentini, and D. R. Tauritz, "Regression testing for model transformations: A multi-objective approach," in *Search Based Software Engineering: 5th International Symposium, SSBSE 2013, St. Petersburg, Russia, August 24-26, 2013. Proceedings 5*. Springer, 2013, pp. 209–223.
- [23] P. Pietsch, H. S. Yazdi, and U. Kelter, "Generating realistic test models for model processing tools," in *2011 26th IEEE/ACM International Conference on Automated Software Engineering (ASE 2011)*. IEEE, 2011, pp. 620–623.
- [24] T. Hartmann, A. Moawad, F. Fouquet, and Y. Le Traon, "The next evolution of MDE: a seamless integration of machine learning into domain modeling," *Software & Systems Modeling*, vol. 18, pp. 1285–1304, 2019.
- [25] K. Berramla, E. A. Deba, and A. E. H. Benyamina, "Automatic Generation of Models from Their Metamodels Using Multilayer Perceptron Network," in *MODELSWARD*, 2024, pp. 272–279.